

Pas à pas — Créer des ressources Kubernetes en YAML

Pour chaque type de ressource : le fichier YAML à créer (via nano ou here-document), puis les commandes pour l'appliquer et le vérifier.

Organiser vos fichiers (dossier optionnel)
Un dossier n'est pas obligatoire, mais conseillé pour regrouper vos manifestes

<code>mkdir mes-manifests</code> Crée un dossier pour ranger les fichiers YAML (facultatif).	<code>cd mes-manifests</code> Se place dans ce dossier avant de créer les fichiers.
---	--

<code>nano <fichier>.yaml</code> Ouvre un éditeur en mode terminal. On tape ou colle le contenu, puis Ctrl+O pour enregistrer et Ctrl+X pour quitter.	<code>cat > <fichier>.yaml << 'EOF' ... contenu ... EOF</code> Crée et remplit le fichier en une seule commande (here-document), sans ouvrir d'éditeur.
--	---

1. Pod

Un conteneur unique, sans supervision

pod.yaml

apiVersion: v1

Version de l'API Kubernetes de base utilisée pour cet objet.

kind: Pod

Type de ressource : un Pod, la plus petite unité déployable.

metadata:

Bloc d'identification de l'objet (nom, labels...).

name: mon-pod

Nom unique donné au pod.

labels:

Étiquettes clé/valeur pour identifier l'objet.

app: mon-app

Étiquette utilisée par les sélecteurs de services.

spec:

Spécifie l'état souhaité du pod.

containers:

Liste des conteneurs à exécuter dans le pod.

- name: nginx

Nom donné à ce conteneur.

image: nginx:1.21

Image Docker et version (tag) à utiliser.

ports:

Ports exposés par le conteneur.

- containerPort: 80

Port interne écouté par le conteneur.

Script complet (copier-coller)

apiVersion: v1

kind: Pod

metadata:

name: mon-pod

labels:

app: mon-app

spec:

containers:

- name: nginx

image: nginx:1.21

ports:

- containerPort: 80

Créer et appliquer

kubectl apply -f pod.yaml

Crée le pod à partir du fichier.

kubectl get pods

Vérifie que le pod est bien démarré.

Vérifier

kubectl get pod mon-pod -o wide

Affiche l'IP, le nœud et le statut détaillé.

kubectl describe pod mon-pod

Détaille l'état, les événements et les erreurs.

Gérer

kubectl logs mon-pod

Affiche les journaux du conteneur.

kubectl exec -it mon-pod -- /bin/bash

Ouvre un terminal à l'intérieur du pod.

Supprimer

kubectl delete pod mon-pod

Supprime le pod.

kubectl delete -f pod.yaml

Supprime la ressource décrite dans le fichier.

2. Deployment

Plusieurs répliques supervisées automatiquement

deployment.yaml		
apiVersion: apps/v1	Groupe d'API 'apps', version 1 — gère les Deployments.	Script complet (copier-coller) apiVersion: apps/v1 kind: Deployment metadata: name: mon-app spec: replicas: 3 selector: matchLabels: app: mon-app template: metadata: labels: app: mon-app spec: containers: - name: nginx image: nginx:1.21 ports: - containerPort: 80
kind: Deployment	Type de ressource : un Deployment, qui gère des répliques de pods.	
metadata:	Bloc d'identification de l'objet.	
name: mon-app	Nom unique du déploiement.	
spec:	État souhaité du déploiement.	
replicas: 3	Nombre de pods identiques à maintenir en permanence.	
selector:	Définit quels pods sont gérés par ce déploiement.	
matchLabels:	Les pods doivent porter ces labels pour être pris en charge.	
app: mon-app	Label recherché sur les pods à gérer.	
template:	Modèle utilisé pour créer chaque pod.	
metadata:	Métadonnées appliquées à chaque pod créé.	
labels:	Labels attribués à chaque pod créé.	
app: mon-app	Doit correspondre au selector défini plus haut.	
spec:	Spécification des conteneurs pour chaque pod.	
containers:	Liste des conteneurs du pod.	
- name: nginx	Nom du conteneur.	
image: nginx:1.21	Image Docker et version à utiliser.	
ports:	Ports exposés par le conteneur.	
- containerPort: 80	Port interne écouté par le conteneur.	

Créer et appliquer

kubectl apply -f deployment.yaml Crée le déploiement et ses pods.	kubectl get deployments Vérifie l'état du déploiement.
--	---

Vérifier

kubectl get deployment mon-app -o wide Affiche le nombre de répliques prêtes et à jour.	kubectl describe deployment mon-app Détaille la stratégie de déploiement et les événements.
--	--

Gérer

kubectl scale deployment mon-app --replicas=5 Change le nombre de répliques à la volée.	kubectl rollout restart deployment mon-app Redémarre les pods sans changer la configuration.
--	---

Supprimer

kubectl delete deployment mon-app Supprime le déploiement et ses pods.	kubectl delete -f deployment.yaml Supprime la ressource décrite dans le fichier.
---	---

3. Service (ClusterIP)

Expose les pods à l'intérieur du cluster

service.yaml		
apiVersion: v1	Version de l'API Kubernetes de base utilisée pour cet objet.	Script complet (copier-coller) apiVersion: v1 kind: Service metadata: name: mon-service spec: selector: app: mon-app ports: - protocol: TCP port: 80 targetPort: 80
kind: Service	Type de ressource : un Service, qui expose des pods.	
metadata:	Bloc d'identification de l'objet.	
name: mon-service	Nom unique du service.	
spec:	État souhaité du service.	
selector:	Sélectionne les pods à exposer via leurs labels.	
app: mon-app	Label recherché sur les pods ciblés.	
ports:	Liste des ports gérés par le service.	
- protocol: TCP	Protocole réseau utilisé.	
port: 80	Port exposé par le service.	
targetPort: 80	Port du conteneur vers lequel rediriger le trafic.	

Créer et appliquer

<code>kubectl apply -f service.yaml</code> Crée le service interne au cluster.	<code>kubectl get svc</code> Vérifie l'IP interne (ClusterIP) attribuée.
---	---

Vérifier

<code>kubectl get svc mon-service -o wide</code> Affiche le sélecteur et les ports exposés.	<code>kubectl describe svc mon-service</code> Détaille les endpoints (pods) rattachés au service.
--	--

Gérer

<code>kubectl edit svc mon-service</code> Modifie la configuration du service en direct.	<code>kubectl get endpoints mon-service</code> Vérifie les pods effectivement reliés au service.
---	---

Supprimer

<code>kubectl delete svc mon-service</code> Supprime le service.	<code>kubectl delete -f service.yaml</code> Supprime la ressource décrite dans le fichier.
---	---

4. NodePort

Expose un service sur un port fixe de chaque nœud

nodeport.yaml		
apiVersion: v1	Version de l'API Kubernetes de base utilisée pour cet objet.	Script complet (copier-coller) apiVersion: v1 kind: Service metadata: name: mon-service-nodeport spec: type: NodePort selector: app: mon-app ports: - protocol: TCP port: 80 targetPort: 80 nodePort: 30036
kind: Service	Type de ressource : un Service, qui expose des pods.	
metadata:	Bloc d'identification de l'objet.	
name: mon-service-nodeport	Nom unique du service.	
spec:	État souhaité du service.	
type: NodePort	Type de service : expose un port fixe sur chaque nœud du cluster.	
selector:	Sélectionne les pods à exposer via leurs labels.	
app: mon-app	Label recherché sur les pods ciblés.	
ports:	Liste des ports gérés par le service.	
- protocol: TCP	Protocole réseau utilisé.	
port: 80	Port exposé à l'intérieur du cluster.	
targetPort: 80	Port du conteneur vers lequel rediriger le trafic.	
nodePort: 30036	Port fixe ouvert sur chaque nœud (plage 30000-32767).	

Créer et appliquer

kubectl apply -f nodeport.yaml Crée le service NodePort.	kubectl get svc mon-service-nodeport Vérifie le port exposé (30000-32767).
---	---

Vérifier

kubectl get svc mon-service-nodeport -o wide Affiche le port de nœud attribué.	kubectl describe svc mon-service-nodeport Détaille les endpoints et le mapping des ports.
---	--

Gérer

kubectl edit svc mon-service-nodeport Modifie la configuration du service en direct.	curl http://<IP-du-nœud>:30036 Teste l'accès depuis l'extérieur du cluster.
---	--

Supprimer

kubectl delete svc mon-service-nodeport Supprime le service NodePort.	kubectl delete -f nodeport.yaml Supprime la ressource décrite dans le fichier.
--	---

5. LoadBalancer

Expose un service via une IP externe (cloud)

loadbalancer.yaml

apiVersion: v1

Version de l'API Kubernetes de base utilisée pour cet objet.

kind: Service

Type de ressource : un Service, qui expose des pods.

metadata:

Bloc d'identification de l'objet.

name: mon-service-lb

Nom unique du service.

spec:

État souhaité du service.

type: LoadBalancer

Type de service : demande une IP externe au fournisseur cloud.

selector:

Sélectionne les pods à exposer via leurs labels.

app: mon-app

Label recherché sur les pods ciblés.

ports:

Liste des ports gérés par le service.

- protocol: TCP

Protocole réseau utilisé.

port: 80

Port exposé sur l'IP externe.

targetPort: 80

Port du conteneur vers lequel rediriger le trafic.

Script complet (copier-coller)

apiVersion: v1

kind: Service

metadata:

name: mon-service-lb

spec:

type: LoadBalancer

selector:

app: mon-app

ports:

- protocol: TCP

port: 80

targetPort: 80

Créer et appliquer

kubectl apply -f loadbalancer.yaml

Crée le service et demande une IP externe au fournisseur cloud.

kubectl get svc mon-service-lb

Vérifie l'adresse IP externe (EXTERNAL-IP).

Vérifier

kubectl get svc mon-service-lb -w

Suit en direct l'attribution de l'IP externe.

kubectl describe svc mon-service-lb

Détaille l'état du load balancer et ses endpoints.

Gérer

kubectl edit svc mon-service-lb

Modifie la configuration du service en direct.

kubectl get events --field-selector involvedObject.name=mon-service-lb

Suit les événements liés au provisioning du LB.

Supprimer

kubectl delete svc mon-service-lb

Supprime le service et libère l'IP externe.

kubectl delete -f loadbalancer.yaml

Supprime la ressource décrite dans le fichier.

6. Ingress

Route le trafic HTTP(S) vers plusieurs services

ingress.yaml		
apiVersion: networking.k8s.io/v1	Groupe d'API réseau, version 1 — gère les Ingress.	Script complet (copier-coller) apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: mon-ingress annotations: nginx.ingress.kubernetes.io/rewrite-target: / spec: rules: - host: monapp.exemple.com http: paths: - path: / pathType: Prefix backend: service: name: mon-service port: number: 80
kind: Ingress	Type de ressource : un Ingress, règles de routage HTTP(S).	
metadata:	Bloc d'identification de l'objet.	
name: mon-ingress	Nom unique de l'Ingress.	
annotations:	Options supplémentaires interprétées par le contrôleur Ingress.	
nginx.ingress.kubernetes.io/rewrite-target: /	Réécrit l'URL avant de la transmettre au service cible.	
spec:	État souhaité de l'Ingress.	
rules:	Liste des règles de routage.	
- host: monapp.exemple.com	Nom de domaine ciblé par cette règle.	
http:	Règles de routage HTTP pour cet hôte.	
paths:	Liste des chemins d'URL gérés.	
- path: /	Chemin d'URL à intercepter.	
pathType: Prefix	Type de correspondance : tout ce qui commence par ce chemin.	
backend:	Service vers lequel rediriger le trafic.	
service:	Référence au service cible.	
name: mon-service	Nom du service backend.	
port:	Port du service backend.	
number: 80	Numéro du port cible.	

Créer et appliquer

kubectl apply -f ingress.yaml Crée la règle de routage Ingress.	kubectl get ingress Vérifie que l'hôte et l'adresse sont bien configurés.
--	--

Vérifier

kubectl get ingress mon-ingress -o wide Affiche l'hôte, les chemins et l'adresse.	kubectl describe ingress mon-ingress Détaille les règles de routage et les backends.
--	---

Gérer

kubectl edit ingress mon-ingress Modifie les règles de routage en direct.	kubectl get pods -n ingress-nginx Vérifie que le contrôleur Ingress fonctionne.
--	--

Supprimer

kubectl delete ingress mon-ingress Supprime la règle de routage.	kubectl delete -f ingress.yaml Supprime la ressource décrite dans le fichier.
---	--